

EE4202 Database Systems

FILES, INDEXES AND DATABASE TUNING

FILES

- A set of blocks/sectors is called a **cluster** of blocks/sectors. A cluster is the *smallest unit of disk allocation* for a file (smallest file size). Ex: 4 KB
- A file header contains information about block addresses, structure of records in the file.
- If the address of the block containing the desired record is not known, a **linear search** (Each block is copied to the main memory buffer and searched for the record) of file blocks must be done.
- A linear search will consume in average ceil (b/2) block access when record is found. // records are not in order in attribute.- linear search. Secondary → main, and linear search, retrieve data is difficult.
- Time consumption is less when the number of block access is less.

Time consumption $\propto$ Number of block accesses

FILES

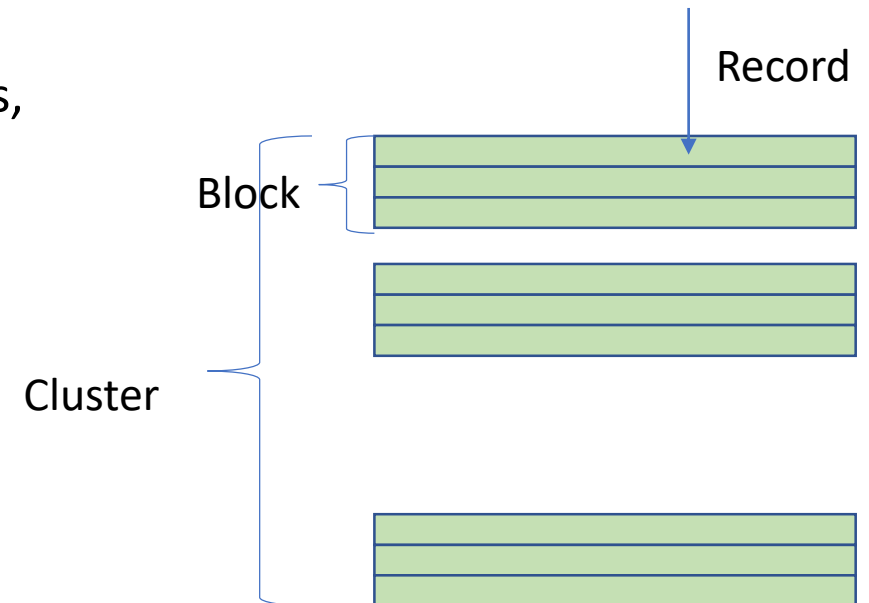
- A tuple in a relation is stored as a record in secondary memory (magnetic disk)
- But, individual tuples are not brought to main memory. The smallest accessible unit of data transfer between main memory and disk is called a **block/sector**. Ex: 512 Bytes
- Several records fit into a block/sector. **Blocking factor (BFr)** is the number of records that fit into a block. Since BFr is an integer, there may be unused space in a block/sector.

$$BFr = \text{floor}\left(\frac{\text{Block Size}(B)}{\text{record size}(R)}\right)$$

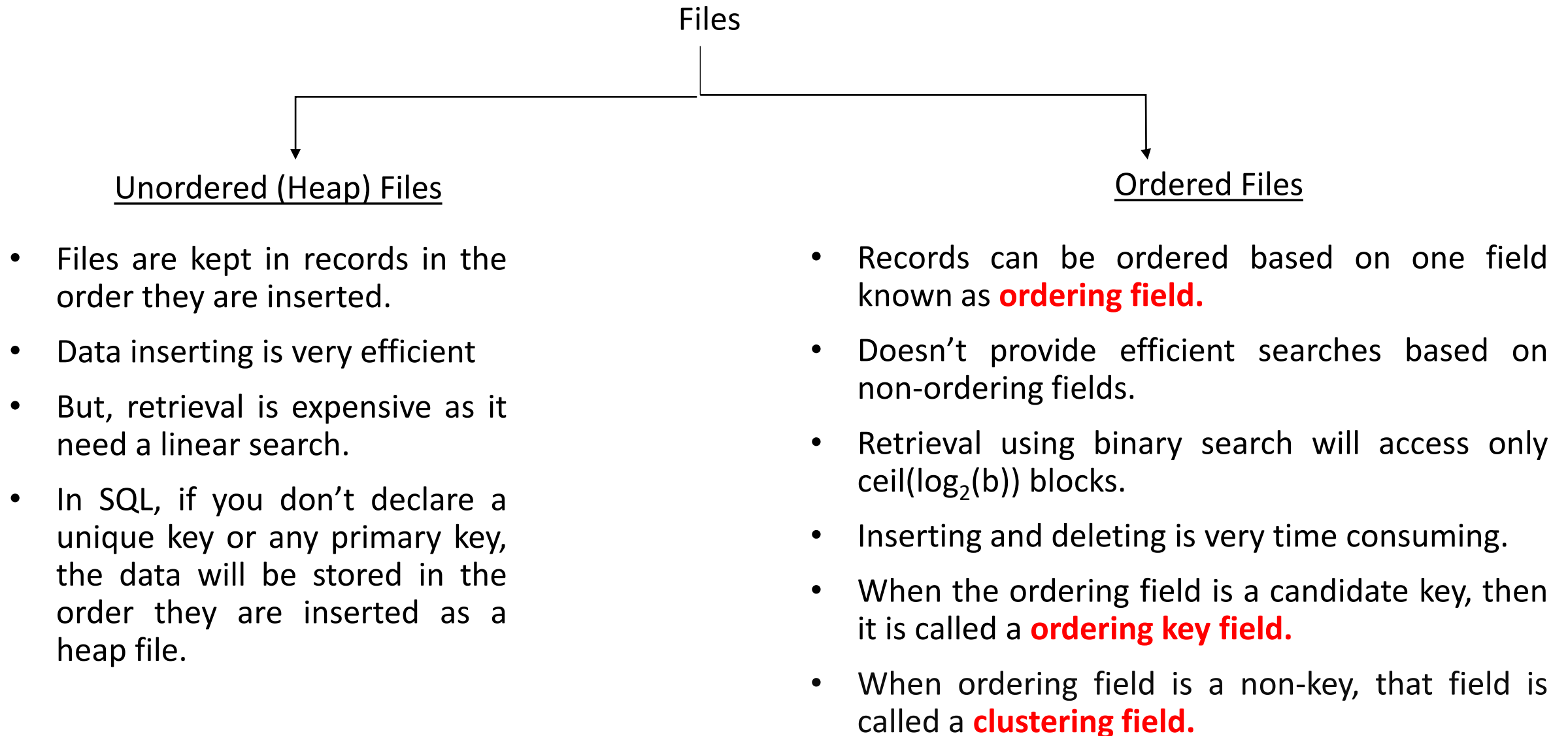
- Floor - smallest integer
- Ex:- $1024/100=10.2$ we can store 10 records to block.
- Hence, the number of blocks (b) need to store r number of records is,

$$b = \text{ceil}\left(\frac{r}{BFr}\right)$$

- ceil - largest integer, if 10.2 we should get 11 blocks.
- To access we need block.
- We should put complete record.

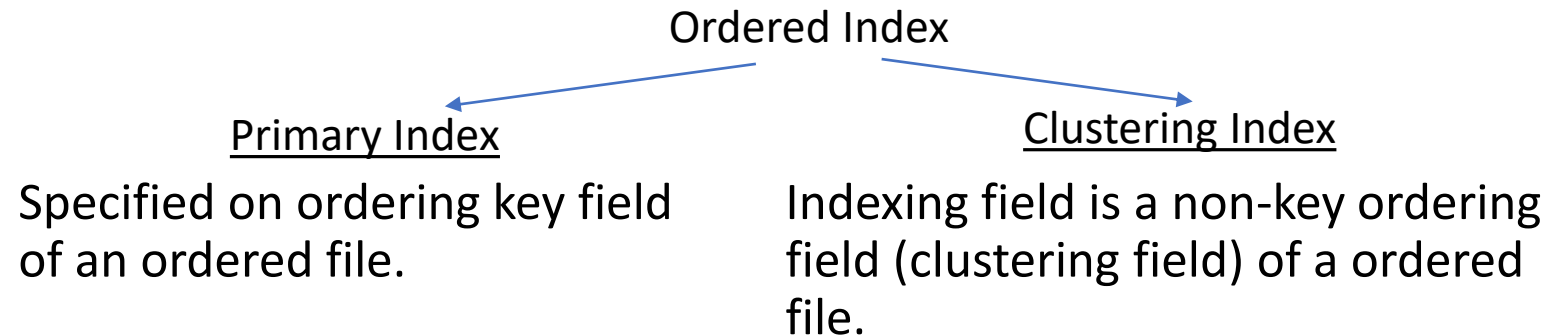


FILES



INDEX

- Indexes are access structures which speed up the retrieval of queries.
- They provide secondary access paths providing alternative ways to access records without affecting physical placement of records in primary data file.
- **Ordered index**: An index which is in a particular order specified for a ordered field of an ordered file. Eg: book index contains index of words with pointers to page numbers.
- A file can have only one physical ordering field.



- An Unordered index is specified on an unordered field of a file.
- A **dense index** has index entries for each and every record of the data file where a **sparse index** is having index entries only for some data file records.

atomic_append.c, 1425
atomicity, 90–92, 465
when accessing shared variables, 631
Austin Common Standards Revision
Group, 13
Autoconf program, 219, 1444
automatic variables, 116, 122
A/UX, 5
awk program, 574, 1437
AXSIG constant, 594

B

B programming language, 2
Bach (1986), 250, 278, 521, 530, 919,
1422, 1437
Bach, M., 1437
background process group, 700, 708, 714
diagram, 701, 717
bad_exclusive_open.c, 90

PRIMARY INDEX

- Index field Ordering key field is usually the primary key of the relation.
- There is only one index entry for each block of the data file (sparse index).
- **Anchor record/block anchor:** Index entry is created for the first record of a block in the data file.
- An Ordered index containing two attributes. The index field and a pointer to record.
- The index field contains primary key of the anchor record and the second field is a pointer to anchor record. Hence, it is a sparse index.
- The index is also stored as a file in disk.
- A binary search on the ordered file needs $\text{ceil}(\log_2(b_f))$ block accesses to find the record.
- A binary search using the index file needs a total of $\text{ceil}((\log_2(b_i) + 1))$ block accesses to find a record in file. That is $\text{ceil}(\log_2(b_i))$ block accesses in index and one additional block access in data file after the index entry pointing block anchor is found.
- In SQL, a primary index is automatically created when a table is created with primary key. The data file is ordered according to primary key.

PRIMARY INDEX EXAMPLE

- ❑ A file contains 30,000 fixed length records of size 100 bytes stored on a disk with sector size 1024 bytes. This file is stored as an ordered file using an ordered key field of size 9 bytes. A block pointer will be 6 bytes of size. If a primary index is created, compare the block accesses with and without the index?

For the ordered file

$$BFr_{file} = \text{floor}\left(\frac{\text{Block Size } (B)}{\text{record size of file } (R_{file})}\right) = \text{floor}\left(\frac{1024}{100}\right) = \text{floor}(10.24) = 10 \text{ records/block}$$

Hence, the number of blocks (b_{file}) need to store r_{file} number of records of file is,

$$b_{file} = \text{ceil}\left(\frac{r_{file}}{BFr_{file}}\right) = \text{ceil}\left(\frac{30,000}{10}\right) = \text{ceil}(3000) = 3000 \text{ blocks}$$

Since the file is ordered; a binary search can be performed.

Number of block accesses to find the record = $\text{ceil}(\log_2(b_{file})) = \text{ceil}(\log_2(3000)) = \text{ceil}(11.55) = 12$ block accesses

PRIMARY INDEX EXAMPLE

- ❑ A file contains 30,000 fixed length records of size 100 bytes stored on a disk with sector size 1024 bytes. This file is stored as an ordered file using an ordered key field of size 9 bytes. A block pointer will be 6 bytes of size. If a primary index is created, compare the block accesses with and without the index?

For the Primary Index

Record size of index = $R_{index} = (\text{ordering key field size} + \text{block pointer size}) = 9 + 6 = 15 \text{ Bytes}$

$$BFr_{index} = \text{floor}\left(\frac{\text{Block/Sector Size (B)}}{\text{record size of index}(R_{index})}\right) = \text{floor}\left(\frac{1024}{15}\right) = \text{floor}(68.27) = 68 \text{ records/block}$$

Since in primary index; there should be one index entry for each block of file;

$$r_{index} = b_{file} = 3000$$

Hence, the number of blocks (b_{index}) need to store r_{index} number of records of the index is,

$$b_{index} = \text{ceil}\left(\frac{r_{index}}{BFr_{index}}\right) = \text{ceil}\left(\frac{3000}{68}\right) = \text{ceil}(44.12) = 45 \text{ blocks}$$

By performing a binary search on the primary index;

$$\text{Number of block accesses to find the record} = \text{ceil}(\log_2(b_{index}) + 1) = \text{ceil}(\log_2(45) + 1) = \text{ceil}(6.49) = 7 \text{ block accesses}$$

CLUSTERING INDEX

- Is a secondary index where index field is a non-key ordering field known as the clustering field.
- The index file contains two attributes. The index field and a pointer to block.
- The index field contains only one index entry for each distinct value of clustering field.
- The second field is a block pointer pointing to first block in the data file that has a value for index field . Hence, it is a sparse index.
- The index is also stored as a file in disk.
- A binary search on the ordered file needs $\text{ceil}(\log_2(b_f))$ block accesses to find the record.
- A binary search using the index file needs a total of $\text{ceil}((\log_2(b_i) + 1))$ block accesses to find a record in file. That is $\text{ceil}(\log_2(b_i))$ block accesses in index and 1 additional block accesses in data file after the pointer to block of data file is found.

SECONDARY INDEX

- Secondary index provides a secondary means of accessing a data file for which some primary access already exists.
- The data file records can be ordered or unordered. Usually secondary index is used in unordered data files called as unordered index.
- Secondary index specified on an ordered non-key field is called clustering index.
- But index field must be ordered irrespective of whether file records are ordered or not.
- The second field is a block pointer or a record pointer.
- If the index field is an unordered candidate key of the file, then index field is called a **secondary key**. In that case there will be index entry for each and every record in the data file. Then, it will be a dense index.
- Index field can also be an unordered non-key value. The index field now will point to intermediate blocks which contain record pointers pointing for duplicates of the index field. The index is now sparse.
- Binary search is not possible in data file if the index field is unordered in the data file. So it will cost $\text{ceil}(b/2)$ disk block accesses without the index.
- A binary search using the index file needs a total of $\text{ceil}((\log_2(b_i) + b_f + b_n))$ block accesses to find a record in file. That is $\text{ceil}(\log_2(b_i))$ block accesses in index and b_f additional block accesses in intermediate block/blocks and b_n blocks in data file.
- $b_f = 0, b_n = 1$ if index field is a secondary key field pointing to records in data file.
- $b_f \geq 1, b_n \geq 1$ if index field is a non key field of the file having a block pointer.
- Insert, delete, update operations are faster than primary index. Searching is slightly slower than primary indices.

SECONDARY INDEX EXAMPLEC

- A file contains 30,000 fixed length records of size 100 bytes stored on a disk with block size 1024 bytes. This file is stored as an unordered file in terms of the secondary index field having 17000 distinct values for the unordered field. A block pointer of index file pointing to intermediate blocks will be 6 bytes of size. If an unordered secondary index is created with an index field of size 9 bytes, compare the block accesses with and without the index for searching the unordered index field. Assume that when using the index there will be in average 1 intermediate block accesses?

For the file

$$BFr_{file} = \text{floor}\left(\frac{\text{Block Size } (B)}{\text{record size of file } (R_{file})}\right) = \text{floor}\left(\frac{1024}{100}\right) = \text{floor}(10.24) = 10 \text{ records/block}$$

Hence, the number of blocks (b_{file}) need to store r_{file} number of records of file is,

$$b_{file} = \text{ceil}\left(\frac{r_{file}}{BFr_{file}}\right) = \text{ceil}\left(\frac{30,000}{10}\right) = \text{ceil}(3000) = 3000 \text{ blocks}$$

Since the file is unordered in terms of the index field; a binary search cannot be performed.

$$\text{Number of block accesses to find the record} = \text{ceil}((b_{file}/2)) = \text{ceil}((3000/2)) = \text{ceil}(1500) = 1500 \text{ block accesses}$$

SECONDARY INDEX EXAMPLE

- A file contains 30,000 fixed length records of size 100 bytes stored on a disk with block size 1024 bytes. This file is stored as an unordered file in terms of the secondary index field having 17000 distinct values for the unordered field. A block pointer of index file pointing to intermediate blocks will be 6 bytes of size. If an unordered secondary index is created with an index field of size 9 bytes, compare the block accesses with and without the index for searching the unordered index field. Assume that when using the index there will be in average 1 intermediate block accesses?

For the Secondary Index

Record size of index = $R_{index} = (\text{ordering key field size} + \text{block pointer size}) = 9 + 6 = 15$ Bytes

$$BFR_{index} = \text{floor}\left(\frac{\text{Block/Sector Size } (B)}{\text{record size of index } (R_{index})}\right) = \text{floor}\left(\frac{1024}{15}\right) = \text{floor}(68.27) = 68 \text{ records/block}$$

Since in secondary index; there should be one index entry for distinct value of the unordered non-key field of the unordered file;

$$r_{index} = 17000$$

Hence, the number of blocks (b_{index}) need to store r_{index} number of records of the index is,

$$b_{index} = \text{ceil}\left(\frac{r_{index}}{BFR_{index}}\right) = \text{ceil}\left(\frac{17000}{68}\right) = \text{ceil}(250) = 250 \text{ blocks}$$

Given that; $b_f = 1$.

$$b_n = \text{ceil}\left(\frac{r_{file}}{\text{Number of distinct values of the index field in file}}\right) = \text{ceil}\left(\frac{30000}{17000}\right) = \text{ceil}(1.76) = 2 \text{ block accesses}$$

By performing a binary search on the secondary index;

$$\text{Number of block accesses to find the record} = \text{ceil}(\log_2(b_{index})) + b_f + b_n = \text{ceil}(\log_2(250) + 1 + 2) = \text{ceil}(10.96) = 11 \text{ block accesses}$$

INDEXING IN SQL

- In SQL; indices are automatically created for the attributes involved in keys. Ex: Primary key, unique (super) keys, foreign keys
- A primary index is created for primary key and file is ordered by primary key. When there is no primary key; the file will be ordered by a candidate key. When there is no primary key or unique key; file is unordered and can be ordered using the keywords **order by**. Ex: ALTER TABLE tablename ORDER BY columnname ASC
- For all other super keys; the indexing field is unordered and secondary indices are created.
- In SQL; the primary index, secondary index with an unordered secondary key field belong to the category of **unique index** as the index field is unique in the original file.
- If there are foreign keys in the table; a **non-unique index** is created automatically as an unordered secondary index with a non-key field value.
- If you create an index on ordered non-key field; a clustering index will be created as non-unique index.
- The default data structure for indices in SQL is binary tree.
 - ✓ Exercise: Visualize the indices in employee table using *show index (show index from employee;)*
 - ✓ Analyze the complexity of the query *select Age from employee where Age=35* using *explain (explain select Age from employee where Age=35;)*
 - ✓ Create a non-unique index on column Age using *create index (create unique index Age_Index using BTREE on employee(Age)) (explain select Age from employee where Age = 26;)* – To drop the index (*drop index Age_Index on employee;*)
 - ✓ Reanalyze the complexity of the same query after creating the non-unique index using *explain******

INDEX TUNING

- Not having an index for a field which is frequently searched in queries is not desirable.
- Using indexes for attributes not frequently searched in queries must be avoided.
- Using an attribute as index-field which undergoes frequent changes such as insert, update, delete will decrease the overall efficiency. *****with purpose
- Use index for a table when a table contains large amount of records.

DESIGN TUNING

- If certain database retrieval often engage in joining of normalized tables; those normalized tables can be denormalized to save the joining time (Normalization increases retrieval time).
- A database table in BCNF having large amount of attributes can be broken down into set of small tables to speed up the queries.

QUERY TUNING GUIDELINES

1. Don't use indices for the attributes retrieved using a nested query. Ex: `Eno IN (select E_number ....)`
2. Don't use indices when the search criteria contains arithmetic expressions (`salary +10 < 100`), NULL comparison (`is NULL`), string comparison (`LIKE %%`)
3. *Keyword DISTINCT causes sorting the file and retrieving. So, only use where appropriate. Avoid at all other times.*
4. Try to avoid intermediate result tables (using views to store intermediate results) by nesting queries in a single query for uncorrelated nested queries
5. For correlated nested queries, intermediate tables are recommended.
6. *When joining tables; joining using one or more attribute having an index is preferred.*
7. Using indices for attributes in multiple selection conditions connected via OR is not encouraged as the query optimizer may not use the indices.

Ex: `(Age >30) OR (Salary < 500)`

QUERY TUNING GUIDELINES

But, Indices can be used for each search condition by splitting the Conditions connected by OR into separate queries connected by UNION operation

Ex: (select Dno from Employee where Age>30) UNION (select Dno from Employee where salary<500);

8. Try to avoid not conditions. Eg: Replace D_no !< 10 by D_no >= 10.

//some guidelines may not be applicable to some DBMS as query optimizer does the query tuning.